
◆ 1. What is the Task Class in C#?

The **Task class** (introduced in .NET 4.0 under **Task Parallel Library – TPL**) represents an **asynchronous operation**.

👉 Namespace:

using System.Threading.Tasks;

👉 Key idea:

A Task is not a thread — it is a **unit of work** that is scheduled and executed by the **Thread Pool**.

◆ 2. Why Task was Introduced?

Before Task, developers used:

- Thread
- ThreadPool
- BackgroundWorker

Problems:

- Hard to manage threads manually
- Poor exception handling
- Difficult to scale

✅ Task solves these by:

- Using thread pool internally
 - Providing async/await support
 - Better exception handling
-

◆ 3. Basic Example of Task

using System;

using System.Threading.Tasks;

```

class Program
{
    static void Main()
    {
        Task task = Task.Run(() =>
        {
            Console.WriteLine("Task is running...");
        });

        task.Wait(); // Wait for completion

        Console.WriteLine("Main thread completed");
    }
}

```

◆ 4. Types of Tasks

✓ 1. Non-Returning Task

```

Task task = Task.Run(() => {
    Console.WriteLine("No return value");
});

```

✓ 2. Returning Task (Task<T>)

```

Task<int> task = Task.Run(() => {
    return 10 + 20;
});

```

```

Console.WriteLine(task.Result); // 30

```

◆ 5. Creating Tasks (Different Ways)

◆ 1. Task.Run (Recommended)

```
Task.Run(() => Console.WriteLine("Running task"));
```

◆ 2. Task.Factory.StartNew

```
Task.Factory.StartNew(() => Console.WriteLine("StartNew"));
```

◆ 3. Using Constructor

```
Task task = new Task(() => Console.WriteLine("Manual task"));
```

```
task.Start();
```

◆ 6. Important Task Properties

Property	Description
IsCompleted	Task finished or not
IsFaulted	Task has exception
IsCanceled	Task canceled
Status	Current state

◆ 7. Task Methods

✓ Wait()

```
task.Wait();
```

✓ Result (for Task)

```
int result = task.Result;
```

◆ 8. Async & Await with Task (Modern Approach)

```
static async Task Main()
{
    int result = await GetData();
    Console.WriteLine(result);
}
```

```
static async Task<int> GetData()
{
    await Task.Delay(1000);
    return 100;
}
```

👉 Benefits:

- Non-blocking
- Cleaner code
- Scalable

◆ 9. Task Lifecycle

1. Created
2. Scheduled
3. Running
4. Completed / Faulted / Canceled

◆ 10. Exception Handling in Task

```
try
{
    Task task = Task.Run(() =>
```

```

{
    throw new Exception("Error");
});

task.Wait();
}
catch (AggregateException ex)
{
    Console.WriteLine(ex.InnerException.Message);
}

```

◆ 11. Task vs Thread (Core Difference)

Now the most important part 🙋

Feature	Task	Thread
Level	High-level abstraction	Low-level
Managed by	Thread Pool	OS
Creation cost	Low	High
Performance	Faster	Slower
Return value	Yes (Task<T>)	No
Async support	Yes (async/await)	No
Exception handling	Easy	Difficult
Scaling	Automatic	Manual
Continuations	Supported	Not supported

Feature	Task	Thread
Cancellation	Built-in	Manual

◆ 12. Example: Thread vs Task

◆ Using Thread

using System.Threading;

```
Thread thread = new Thread(() =>
{
    Console.WriteLine("Thread running");
});
thread.Start();
thread.Join();
```

◆ Using Task

```
Task.Run(() => Console.WriteLine("Task running")).Wait();
```

◆ 13. Key Conceptual Difference

👉 Thread = Worker (physical execution unit)

👉 Task = Work (logical unit)

Think like this:

- Thread → Employee
 - Task → Job assigned to employee
-

◆ 14. When to Use Task vs Thread

✅ Use Task (Recommended in 95% cases)

- Async programming
- I/O operations
- Parallel processing
- Web APIs
- Background jobs

 Use Thread only when:

- Need full control over thread
 - Long-running dedicated operations
 - Real-time systems
-